

Case Study 09 — Beer-Lambert Tuning

Reference concentration and path length

Contents

1 Case Study 09 — Beer-Lambert Tuning	1
1.1 What you will produce	1
1.2 1. Prerequisites	1
1.3 2. Inputs	1
1.4 3. Run with the CLI	2
1.5 4. Run with the HTTP API	2
1.6 5. Drive both interfaces from one Python script	2
1.7 6. Expected output layout	2
1.8 7. Related case studies	3

1 Case Study 09 — Beer-Lambert Tuning

Change the **reference concentration** and **path length** used in the Beer-Lambert step. Both knobs enter the absorbance only through their product, $c \cdot L$. Here we raise the concentration by a factor of 10 ($1\text{e-}5 \text{ M} \rightarrow 1\text{e-}4 \text{ M}$) and shorten the path by a factor of 10 ($1.0 \text{ cm} \rightarrow 0.1 \text{ cm}$), so the product stays the same.

The user-selectable knobs are:

- CLI: `--concentration-m FLOAT (mol L-1)` and `--path-cm FLOAT (cm)`
- API: form fields `concentration_m=FLOAT` and `path_cm=FLOAT`

The defaults are $1\text{e-}5 \text{ M}$ and 1.0 cm , giving a product of $1 \times 10^{-5} \text{ mol L}^{-1} \cdot \text{cm}$ that keeps `absorbance_max` below ~ 0.9 for most of the bundled dyes.

1.1 What you will produce

Everything is written under one `output/` directory inside this case study folder:

Folder	By	Contents
<code>output/cli/tables/</code>	CLI	Result tables (CSV / JSON / XLSX)
<code>output/cli/plots/</code>	CLI	TIFF plots per sample, light source, and (sample, light) pair
<code>output/api/</code>	API	Same <code>tables/</code> and <code>plots/</code> tree, extracted from the ZIP
<code>output/api/analysis_outputs.zip</code>	API	Raw ZIP response

Light sources used: **AM1.5G** + **LED-B4**.

1.2 1. Prerequisites

If you have never run any case study before, follow [Case Study 01 §1 Prerequisites](#) once to install the package, the `requests` library, and either the Python or Docker option for the HTTP API. Every case study uses the same environment.

1.3 2. Inputs

The case-study `input.json` lists 2 Gaussian TD-DFT outputs. The manifest uses relative paths that point at the bundled data under `../../input/files/`, so no files are copied or duplicated. To swap in your own data, replace `input.json` with a manifest of the same shape; you do not need to touch `submit.py`.

1.4 3. Run with the CLI

Open a terminal at the repository root, then run a single command:

```
overlap-calculator analyze --input case_studies/09_beer_lambert_tuning/input.json --out
→ case_studies/09_beer_lambert_tuning/output/cli --default-light-sources AM15G,LEDB4 --concentration-m
→ 1e-4 --path-cm 0.1 --plot-dpi 400
```

After the run, `case_studies/09_beer_lambert_tuning/output/cli/` contains a `tables/` directory (and a `plots/` directory unless plots were disabled). The `--default-light-sources` flag pins the bundled sources to AM1.5G + LED-B4; remove it to compare against the full default set (AM15G,LEDB4,LEDB2,LEDB3,CIEFL10).

1.5 4. Run with the HTTP API

Start the API as documented in [Case Study 01 §1](#) and verify with `curl http://localhost:8000/health` (or `:8080` if you started the API via `docker compose`). Then submit the same uploads in one `curl` command:

```
curl -X POST http://localhost:8000/analyze -F "files=@input/files/slurm-5473089.out" -F
→ "files=@input/files/slurm-5473178.out" -F "default_light_sources=AM15G,LEDB4" -F
→ "concentration_m=1e-4" -F "path_cm=0.1" -F "plot_dpi=400" --output
→ case_studies/09_beer_lambert_tuning/output/api/analysis_outputs.zip
```

Each `-F "files=@..."` is one upload. The form fields mirror the CLI flags. The response is a ZIP archive saved as `output/api/analysis_outputs.zip`.

1.6 5. Drive both interfaces from one Python script

Instead of typing the commands above, run `submit.py` from the repository root:

```
python case_studies/09_beer_lambert_tuning/submit.py
```

The script reads `input.json`, runs the CLI step (writing to `output/cli/`), then POSTs the same uploads to `/analyze` and unpacks the response into `output/api/`. If the API is not reachable, it prints a hint and exits cleanly without failing the CLI step.

1.7 6. Expected output layout

```
case_studies/09_beer_lambert_tuning/output/
+-- cli/
|   +-- tables/results.{csv,json,xlsx}      (2 samples x 2 light sources = 4 rows (c = 1e-4 M, L = 0.1
→ cm))
|   +-- tables/results_timings.{csv,json,xlsx}
```

```

|   +-- tables/descriptor_summary.{csv,xlsx}
|   +-- tables/ranking_by_light_source__<metric>.{csv,json,xlsx}   (gaussian/lorentzian x
↪   absorbed_fraction/shape_overlap)
|   +-- tables/ranking_by_sample__<metric>.{csv,json,xlsx}         (same four metric variants)
|   +-- plots/                                                       (also
↪   plots/ranking/by_light_source/<metric>/ and plots/ranking/by_sample/<metric>/)
+-- api/
    +-- analysis_outputs.zip
    +-- tables/...
    +-- plots/...

```

API sample IDs are auto-generated from each upload’s %chk header, while the CLI uses the `sample_id` you set in `input.json`. This is the only cosmetic difference between the two trees; the tables align row by row otherwise.

1.8 7. Related case studies

- [Case Study 08 — Broadening Sigma](#)
- [Case Study 10 — Wavelength Grid](#)
- [Case Study 01 — Theoretical Only](#)